



Object-Oriented Software Engineering



First Edition, WCB/McGraw-Hill, 2008



UNIVERSITY OF THE FREE STATE • UNIVERSITEIT VAN DIE VRYSTAAT • YUNIVESITHI YA FREISTATA

P O Box 339, Internal Box 65, Bloemfontein, 9300 • Republic of South Africa

Tel: +27 (0)51-401 2745 • Fax: +27 (0)51-401 2754

CHAPTER 4

TEAMS

4.1 Team Organization

- Most products are too large to be completed by a single software professional within the given time constraints
- As a result, the product may be given to a group of professionals organised as a *team*.

Team Organization: Task Sharing

- Some tasks can be shared and others must be done individually
- If one farm hand can pick a strawberry field in 10 days, ten farm hands can pick the same strawberry field in 1 day
- One elephant can produce a calf in 22 months, but 22 elephants cannot possibly produce that calf in 1 month

Team Organization: Task Sharing (contd)

- Unlike elephant production, it is possible to share coding tasks between members of a team
- Unlike strawberry picking, team members must interact in a meaningful and effective way

Team Organization: Task Sharing (contd)

- Example:
 - Sheila and Harry code two modules, m_1 and m_2
- What can go wrong?
 - Both Sheila and Harry may code m_1 , and ignore m_2
 - Sheila may code m_1 , Harry may code m_2 . When m_1 calls m_2 it passes 4 parameters; but m_2 requires 5 parameters
 - Or, the order of parameters in m_1 and m_2 may be different
 - Or, the order may be same, but the data types may be slightly different
- This has nothing whatsoever to do with technical competency
 - Team organization is a managerial issue

Team Organization: Communications Problems

- Example
 - There are three channels of communication between the three programmers working on a project. The deadline is rapidly approaching but the code is not nearly complete
- “Obvious” solution:
 - Add a fourth programmer to the team

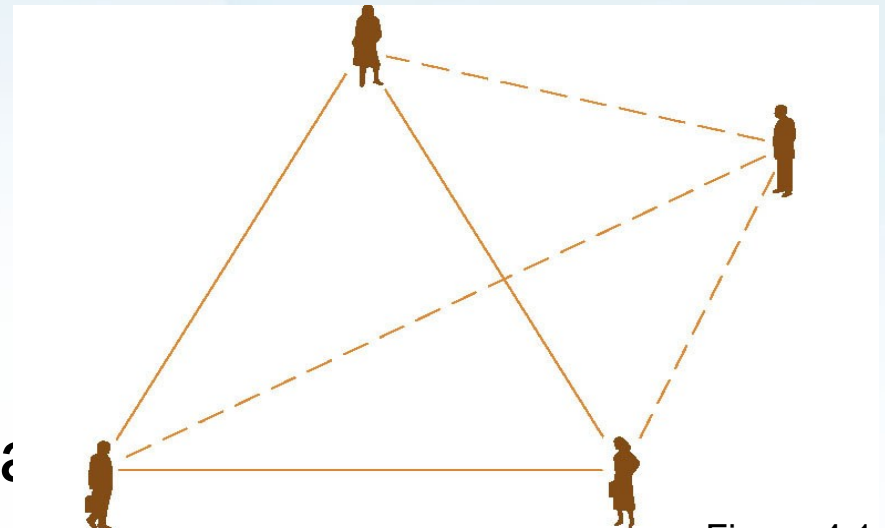


Figure 4.1

Team Organization: Communications Problems

- But other three have to explain in detail
 - What has been accomplished
 - What is still incomplete
- Brooks's Law
 - Adding additional programming personnel to a team when a product is late has the effect of making the product even later

Team Organization

- Teams are used throughout the software production process
 - But especially during implementation
 - Here, the discussion is presented within the context of programming teams
- Two extreme approaches to team organization
 - Democratic teams (Weinberg, 1971)
 - Chief programmer teams (Brooks, 1971; Baker, 1972)

4.2 Democratic Team Approach

- Basic underlying concept — *egoless programming*
- Programmers can be highly attached to their code
 - They even name their modules after themselves
 - They see their modules as extension of themselves
- If a programmer sees a module as an extension of his/her ego, he/she is not going to try to find all the errors in “his”/“her” code
 - If there is an error, it is termed a *bug* 🐛

Democratic Team Approach

- Proposed solution
- Egoless programming
 - Restructure the social environment
 - Restructure programmers' values
 - Encourage team members to find faults in code
 - A fault must be considered a normal and accepted event
 - The team as whole will develop a culture, a group identity
 - Modules will “belong” to the team as whole
 - A group of up to 10 egoless programmers constitutes a *democratic team*

Democratic Team Approach: Difficulties

- Management may have difficulties
 - Democratic teams are hard to introduce into an undemocratic environment
 - Democratic teams work for a common cause with no identifiable leader
 - Do not try to get promotion
 - Managers may have difficulty accepting egoless programming

4.2.1 Analysis of democratic team approach

- Strengths of Democratic Team Approach:
 - They have a positive attitude towards finding faults
 - Democratic teams are enormously productive
 - Work best when the problem is difficult
 - Function well in a research environment
 - Leads to more rapid detection of faults and higher-quality code
- Problems with Democratic Team Approach:
 - Managers may have difficulty accepting egoless programming
 - Programmers may resent others finding fault with their code
 - Democratic teams have to spring up spontaneously

4.3 Chief Programmer Team Approach

- Consider a 6-person team
 - Fifteen 2-person communication channels
 - This team cannot do 6 person-months of work in 1 month

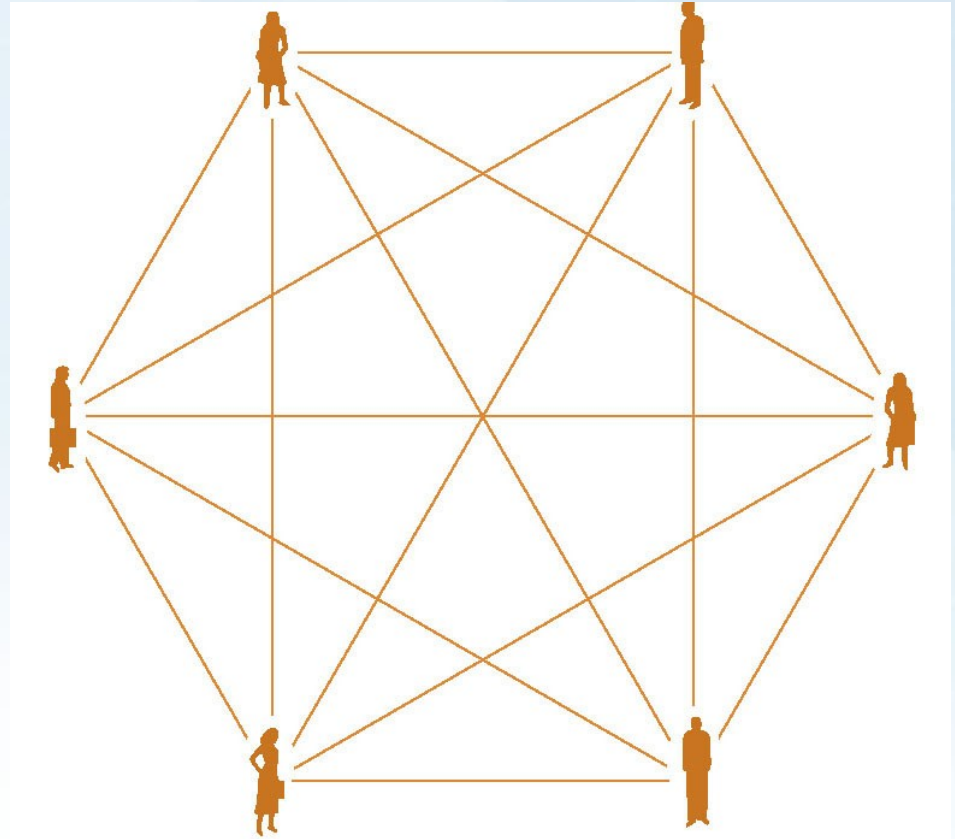


Figure 4.2

Chief Programmer Team Approach

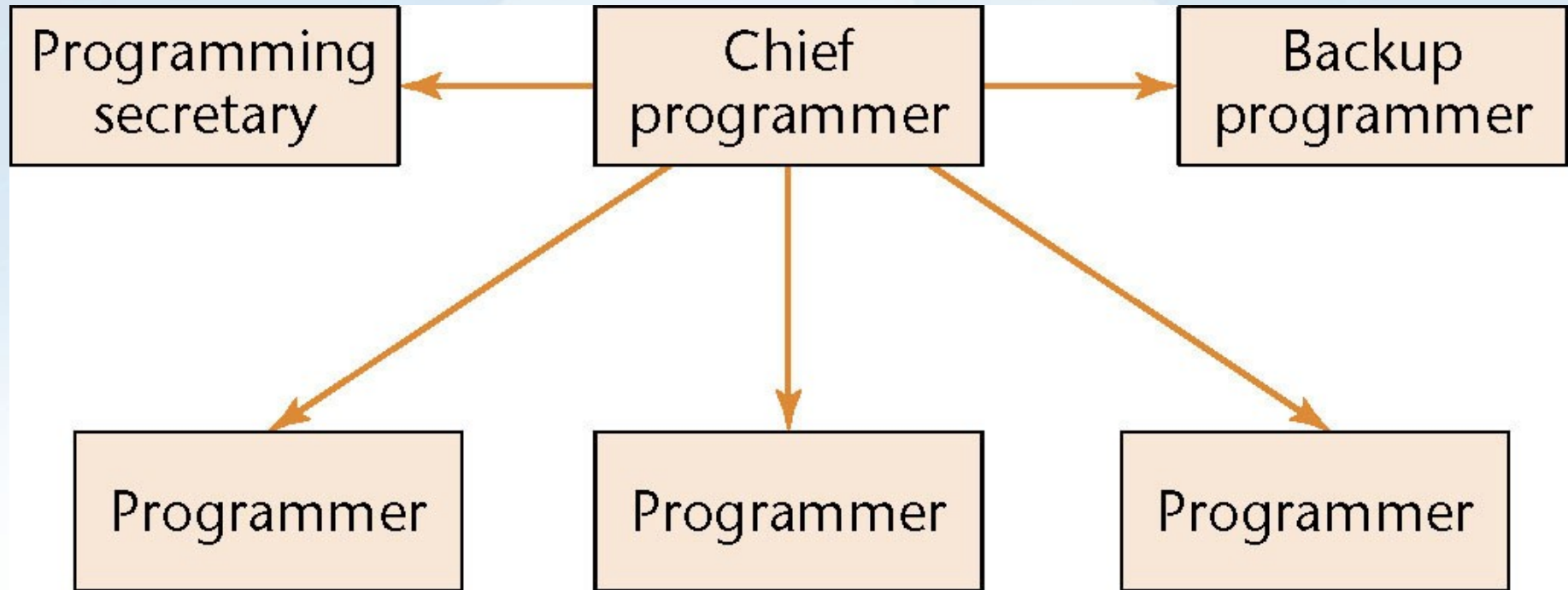


Figure 4.3

- Six programmers, but now only 5 lines of communication

Chief Programmer Team Approach

- Basic idea behind the concept of the chief programmer team
 - Analogy: chief surgeon directing an operation, assisted by
 - Other surgeons
 - Nurses
 - Other experts, such as cardiologists
- Key aspects of chief programmer team
 - Specialisation
 - Hierarchy

Chief Programmer Team Approach:

Team structure

- Chief programmer
 - Successful manager *and* highly skilled programmer
 - Does the architectural design
 - Allocates coding among the team members
 - Writes the critical (or complex) sections of the code
 - Handles all the interfacing issues
 - Reviews the work of the other team members
 - Is personally responsible for every line of code

Chief Programmer Team Approach: Team structure

- Back-up programmer
 - Necessary only because the chief programmer is human and could therefore become ill, etc.
 - The back-up programmer must be in every way as competent as the chief programmer, and
 - Must know as much about the project as the chief programmer
 - The back-up programmer does black-box test case planning and other tasks that are independent of the design process

Chief Programmer Team Approach: Team structure

- Programming secretary
 - A highly skilled, well paid, central member of the chief programmer team
 - Responsible for maintaining the program production library (documentation of the project).

Chief Programmer Team Approach: Team structure

- Programmers
 - Do nothing but program
 - All other aspects are handled by the programming secretary

4.3.2 Impracticality of the Chief Programmer Team Approach

- The chief programmer must be a highly skilled programmer *and* a successful manager
- There is a shortage of highly skilled programmers
- There is a shortage of successful managers
- The qualities needed to be a highly skilled programmer are unlikely to be found in a successful manager, and vice versa

Impracticality of the Chief Programmer Team Approach

- The *back-up programmer* must be as good as the chief programmer
 - But he/she must take a back seat (and a lower salary) waiting for something to happen to the chief programmer
 - Top programmers, top managers will not do that
- The *programming secretary* does nothing but paperwork all day
 - Software professionals hate paperwork
- Therefore, chief programmer teams are impractical

4.4 Beyond Chief Programmer and Democratic Teams

- We need ways to organize teams that
 - Make use of the strengths of democratic teams and chief programmer teams, and
 - Can handle teams of 20 (or 120) programmers
- A strength of democratic teams
 - A positive attitude to finding faults
- Use chief programmer team in conjunction with code reviews

Beyond Chief Programmer and Democratic Teams

- Potential pitfall:
 - The chief programmer is personally responsible for every line of code
 - He/she must therefore be present at reviews
 - The chief programmer is also the team manager
 - Reviews should not be used for performance appraisal
 - He/she must therefore *not* be present at reviews!

Beyond Chief Programmer and Democratic Teams

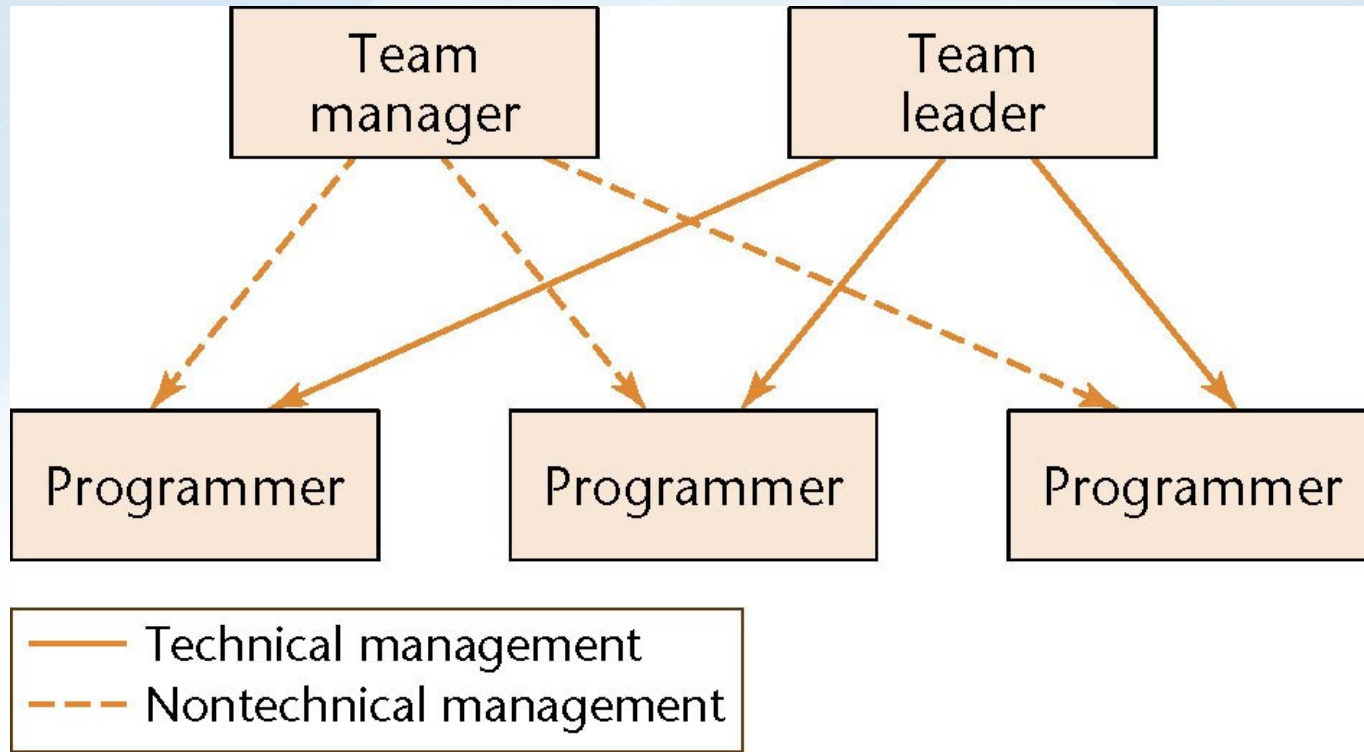


Figure 4.4

- Solution
 - Reduce the managerial role of the chief programmer

Beyond Chief Programmer and Democratic Teams

- Chief programmer should be replaced by:
 - Team leader
 - In charge of the technical aspects of the team's activities
 - Team manager
 - Responsible for all nontechnical managerial decisions
- The team leader is responsible for only technical management

Beyond Chief Programmer and Democratic Teams

- Budgetary and legal issues, and performance appraisal are not handled by the team leader
- The team leader participates in reviews — the team manager is not permitted to do so

Beyond Chief Programmer and Democratic Teams: Larger Projects

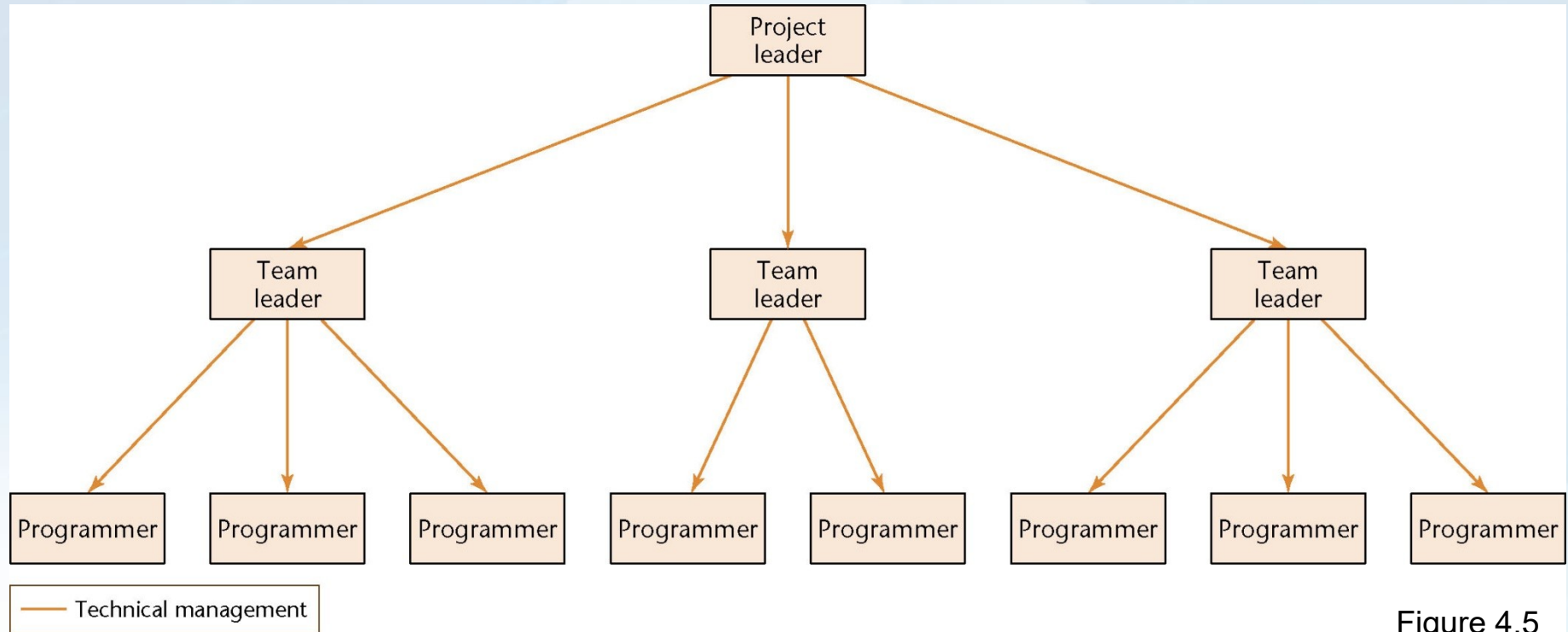


Figure 4.5

- The nontechnical side is similar
 - For even larger products, add additional layers

4.5 Synchronize-and-Stabilize Teams

- Used by Microsoft
- Windows
 - 30 million LOC
 - 3000 programmers and testers
- Products consist of 3 or 4 sequential builds
- Small parallel teams
 - Led by a manager
 - 3 to 8 developers
 - 3 to 8 testers (work one-to-one with developers)
 - The team is given the overall task specification
 - They may design the task as they wish

Synchronize-and-Stabilize Teams

- Strength
 - Individual programmers are encouraged to be creative and innovative, like democratic team
 - Daily synchronisation ensures hundreds of developers work together toward a common goal
- Rules
 - Programmers must adhere to the time for entering the code into the database for that day's synchronization
- Analogy
 - Letting children do what they like all day...
 - ... but with a 9 P.M. bedtime

Synchronize-and-Stabilize Teams

- Will this work in all companies?
 - Perhaps if the software professionals are as good as those at Microsoft

4.6 Teams For Agile Processes

- Feature of agile processes
 - All code is written by two programmers sharing a computer
 - “Pair programming”

Teams For Agile Processes: Strengths of Pair Programming

- Programmers should not test their own code
 - One programmer draws up the test cases, the other implements the code using the test cases
- If one programmer leaves, the other is sufficiently knowledgeable to continue working with another pair programmer
- An inexperienced programmer can learn from his or her more experienced team member
- Computers in the middle of a large room promote group ownership of code (egoless programming)

4.7 Open-Source Programming Teams

- Open-source projects
 - Are generally staffed by teams of unpaid volunteers
 - Who communicate asynchronously (via e-mail)
 - With no team meetings and
 - With no managers
 - There are no specifications or designs, and
 - Little or no other documentation

Open-Source Programming Teams

- Individuals volunteer to take part in an open-source project for two main reasons
 - Reason 1: For the sheer enjoyment of accomplishing a worthwhile task
 - In order to attract and keep volunteers, they have to view the project as “worthwhile” at all times
 - Reason 2: For the learning experience
- Software professionals often join an open-source project to gain new skills
 - For a promotion, or
 - To get a better job elsewhere
- Many employers view experience with a large, successful open-source project as better than additional academic qualifications

Open-Source Programming Teams

- The members of the open-source team must at all times feel that they are making a contribution
- For all these reasons, it is essential that the key individual behind an open-source project be a superb motivator
 - Otherwise, the project is doomed to inevitable failure

Open-Source Programming Teams

- For a successful open-source project, the members of the core group must be top-caliber individuals with skills of the highest order
- Such top-class individuals can thrive in the unstructured environment of an open-source team

Choosing an Appropriate Team Organization

- There is no one solution to the problem of team organization
- The “correct” way depends on
 - The product
 - The outlook of the leaders of the organization
 - Previous experience with various team structures